

Security Audit

Kingdomly 3rd (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	22
Conclusion	23
Our Methodology	24
Disclaimers	26
About Hashlock	27

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Kingdomly team partnered with Hashlock to conduct a security audit of their Payout.sol, KingdomlyFeeContract.sol, and KingdomlyStaking.sol smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Kingdomly is a platform that enables users to create and manage NFT collections with ease. It provides tools for launching generative or unique NFTs without requiring technical skills. The platform handles the entire NFT creation process, including smart contracts and metadata, and offers features like customizable mint pages, private sales, and token-gated applications. Kingdomly aims to simplify NFT creation, allowing creators to focus on their art and community engagement.

Project Name: Kingdomly

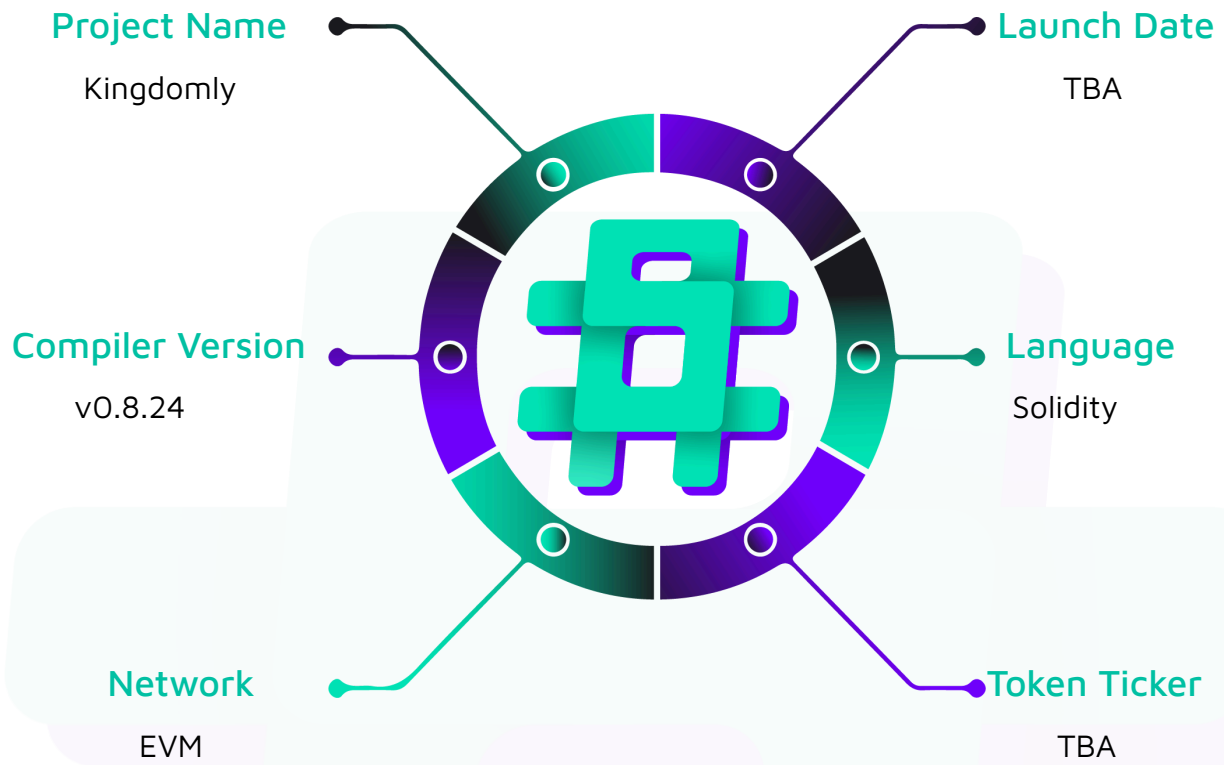
Project Type: Defi

Compiler Version: 0.8.24

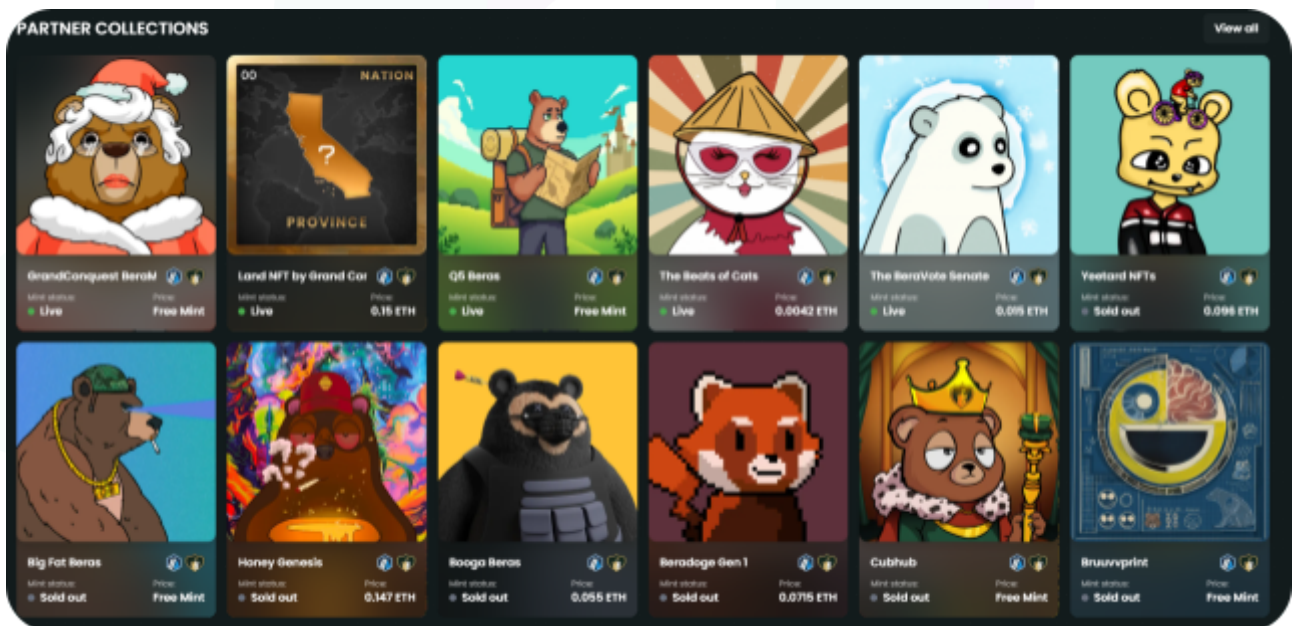
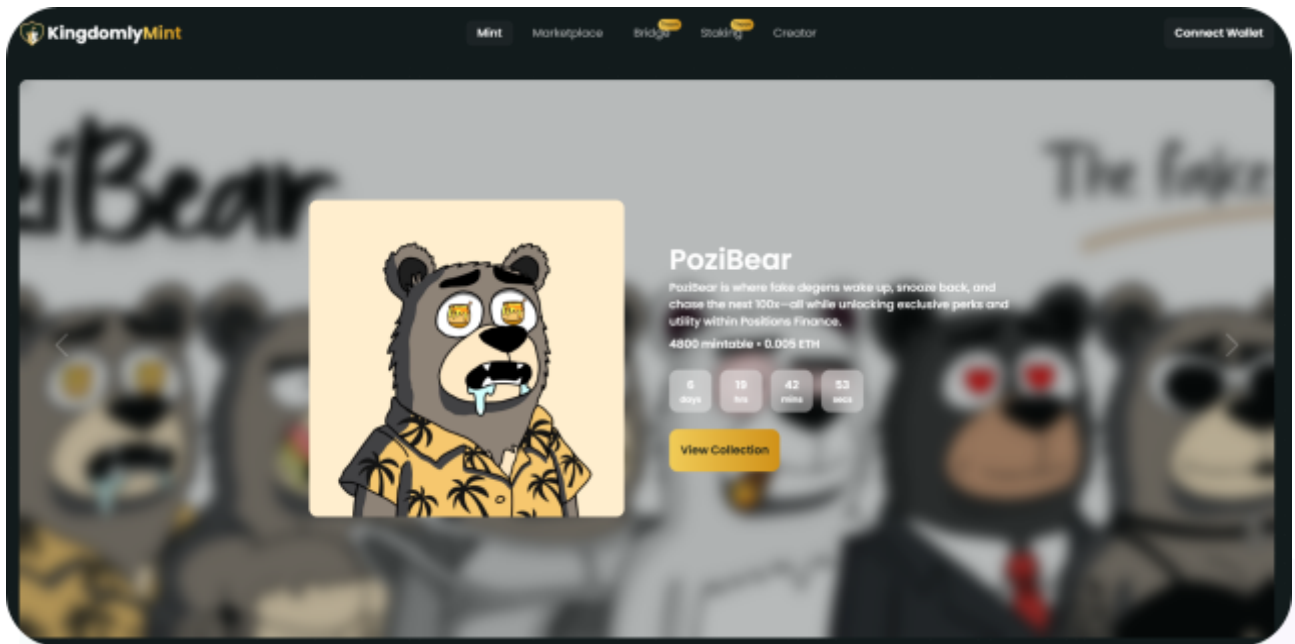
Website: www.kingdomly.app

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Kingdomly project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Kingdomly Smart Contracts
Platform	EVM / Solidity
Audit Date	January, 2025
Contract 1	Payout.sol
Contract 1 MD5 Hash	ade99b486b4b5a9853d0417ef85a870d
Contract 2	KingdomlyFeeContract.sol
Contract 2 MD5 Hash	11003f837af0666869505c38a5f16526
Contract 3	KingdomlyStaking.sol
Contract 3 MD5 Hash	264711dc67399c91948bde80db8d362e

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

2 Medium severity vulnerabilities

2 Low severity vulnerabilities

1 Gas Optimisation

3 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Payout.sol - Allows owner to: - Add/Remove payout addresses - Payout	Contract achieves this functionality.
KingdomlyFeeContract.sol - Allows owner to: - Update the max price age - Update oracle and get one dollar in wei	Contract achieves this functionality.
KingdomlyStaking.sol - Allows owner to: - Set a reward token - Set the reward redemption process - Set multipliers for specific users - Set multipliers for specific tokens - Add a points boost to specific users - Allows users to: - Stake their NFTs from a supported collection - Withdraw their staked NFTs from the contract - Redeem their accumulated rewards based on their staked NFTs	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Kingdomly project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Kingdomly project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] KingdomlyStaking#redeemRewards - Lack of `hasRedeemed[msg.sender]` check allows users to claim an unlimited amount of rewards

Description

The `redeemRewards` function is supposed to be called once per a user after the redemption period.

However, the function is missing checking whether the caller has already redeemed or not.

Vulnerability Details

The reward amount to be redeemed is calculated based on the users points and the points are being kept as long as they stake NFTs.

`hasRedeemed[msg.sender]` is updated to true in the function to prevent users to redeem multiple times but the function doesn't check if this value is already true at the beginning of the function.

This could allow users to call the function multiple times and claim an unlimited amount of rewards.

```
function redeemRewards() external nonReentrant rewardRedemptionActive(true) {
    uint256 totalUserPoints = getTotalUserPoints(msg.sender);

    storedPoints[msg.sender] = 0;
    redeemedPoints[msg.sender] = totalUserPoints;

    uint256 userRewardPercentage = getUserRedeemedPointsPercentage(msg.sender);

    // Loop through all reward tokens and distribute rewards
```

```

    for (uint256 i = 0; i < rewardTokens.length; i++) {
        IERC20 token = rewardTokens[i];
        uint256 depositedAmount = depositedTokenAmounts[address(token)];
        ...
        if (depositedAmount > 0) {
            uint256 tokenRewardAmount = (depositedAmount * userRewardPercentage) /
1_000_000;

            if (tokenRewardAmount > 0) {
                token.safeTransfer(msg.sender, tokenRewardAmount);
                ...
            }
        }
    }
    hasRedeemed[msg.sender] = true;
    ...
}

```

Proof of Concept

An example of a test that simulates allowing users to redeem multiple times is shown below.

```

it("Users are allowed to redeem rewards multiple times", async () => {
    const { kingdomlyStaking, testFT, testNft, staker_1, staker_2, nftOwner } =
        await loadFixture(deployKingdomlyStakingFixture);
    const stakeFee = await kingdomlyStaking.getStakeFee();
    const depositAmount = ethers.parseEther("1000");
    // Setup staking
    await testNft.connect(staker_1).setApprovalForAll(
        await kingdomlyStaking.getAddress(),
        true
    );
    await testNft.connect(staker_2).setApprovalForAll(
        await kingdomlyStaking.getAddress(),
        true
    );
    // Stake NFTs
    await kingdomlyStaking.connect(staker_1).stake(
        await testNft.getAddress(),

```

```

    [0],
    { value: stakeFee }
  );
  await kingdomlyStaking.connect(staker_2).stake(
    await testNft.getAddress(),
    [10],
    { value: stakeFee }
  );
  // Deposit rewards
  await testFT.mint(nftOwner.address, depositAmount);
  await testFT.approve(await kingdomlyStaking.getAddress(), depositAmount);
  await kingdomlyStaking.depositRewardToken(
    await testFT.getAddress(),
    depositAmount
  );
  // Move time forward and start redemption
  await time.increase(24 * 60 * 60);
  const stakersAddresses = await kingdomlyStaking.getStakers();
  let pendingPoints = BigInt(0);
  for(let i = 0; i < stakersAddresses.length; i++) {
    pendingPoints += await
kingdomlyStaking.calculateUserCurrentlyStakedTokenPoints(stakersAddresses[i]);
  }
  await kingdomlyStaking.startRewardRedemption(pendingPoints);
  // First redeem by staker_1
  await kingdomlyStaking.connect(staker_1).redeemRewards();
  // Check reward token balances
  expect(await testFT.balanceOf(staker_1.address)).to.equal(
    depositAmount * BigInt(1) / BigInt(2)
  );
  // Second redeem by staker_1
  await kingdomlyStaking.connect(staker_1).redeemRewards();
  // Check reward token balances
  expect(await testFT.balanceOf(staker_1.address)).to.equal(depositAmount);
  // As a result, the staker_1 redeemed all the rewards
});

```

Impact

Anyone can drain all the rewards from the contract.



Recommendation

Check `hasRedeemed[msg.sender]` at the beginning of the function and revert if it's already true.

Status

Resolved

[H-02] KingdomlyStaking - Stake fees are locked in the contract forever**Description**

Users are supposed to pay fees while both staking and withdrawing.

The fee is stored in the contract but the contract doesn't have the function to withdraw the fees.

This could lead to lock the fees in the contract forever.

Impact

The fees are locked in the contract forever.

Recommendation

Add a function which can only be called by the owner to withdraw the fees stored.

Status

Resolved

Medium

[M-01] KingdomlyStaking#calculateUserCurrentlyStakedTokenPoints -

Division Before Multiplication results in precision loss

Description

The `calculateUserCurrentlyStakedTokenPoints` function calculates the point by first dividing the time difference between the current time and staked time by 3600 and then multiplying by `totalMultiplier * collectionDailyPoints[userStakedTokens[i].collection]`.

```
function calculateUserCurrentlyStakedTokenPoints(address _user) public view returns (uint256) {
    ...
    uint256 stakedHours = (calculationTimestamp - userStakedTokens[i].stakedAt) / 3600;
    userCalculatedRewards +=
        (totalMultiplier * collectionDailyPoints[userStakedTokens[i].collection]
        * stakedHours) / 24;
    ...
}
```

However, division before multiplication is not a good solidity design pattern and it could result in precision loss.

Recommendation

It's recommended that first multiplying the time difference by `totalMultiplier * collectionDailyPoints[userStakedTokens[i].collection]` and then dividing by 86400.

```
function calculateUserCurrentlyStakedTokenPoints(address _user) public view returns (uint256) {
    ...
    userCalculatedRewards +=
        (totalMultiplier * collectionDailyPoints[userStakedTokens[i].collection]
        * (calculationTimestamp - userStakedTokens[i].stakedAt)) / 86400;
}
```

```
...
}
```

Status

Resolved

[M-02] Payout#payout, KingdomlyStaking#redeemRewards - Denial of Service risk due to gas limit while handling all payout addresses and reward tokens

Description

Because the Payout contract allows for an unlimited number of payout addresses to be added, when the payout function iterates over the payoutAddresses value while handling all payout addresses, the contract may revert since the cost could exceed the block gas limit if the array grows enough.

This issue also exists in the redeemRewards function while handling all the reward tokens from the rewardTokens array.

Vulnerability Details

The payout function loops over all payout addresses to be handled:

```
for (uint256 i = 0; i < payoutAddresses.length; i++) {
    ...
}
```

Depending on the number of payout addresses that the contract supports, calls to the payout function may run out of gas, resulting in a denial of service of all external-facing functions.

Recommendation

It's recommended that the payout function allows for the specification of an offset and length so that payout addresses can be handled in batches or limit the number of payout addresses to be added.

And add a limit for the number of reward tokens to be supported in the KingdomlyStaking contract.

Status

Resolved

Low

[L-01] **KingdomlyStaking#depositRewardToken** - Lack of `rewardRedemptionActive` modifier allows the function to be called after the redemption period

Description

The `depositRewardToken` function is supposed to not be called after the redemption period but the function doesn't have any restriction for the requirement.

This could lead to calling the function after the redemption period.

Recommendation

Add `rewardRedemptionActive` modifier in the function.

Status

Resolved

[L-02] **KingdomlyStaking#setNewKingdomlyFeeContract, KingdomlyFeeContract#updateMaxPriceAgeInSeconds** - Lack of emitting events

Description

The above functions are missing events when key storages are updated.

Recommendation

Add relevant events based on the variables to be updated.

Status

Resolved

Gas

[G-01] KingdomlyFeeContract - `pyth` and `ethUsdPriceId` could be made immutable

Description

The `pyth` and `ethUsdPriceId` variables are only updated in the constructor.

Recommendation

Consider making the variables immutable.

Status

Resolved

QA

[Q-01] KingdomlyFeeContract, KingdomlyStaking - Unused error declarations

Description

The following errors are declared but they are not used at all.

```
KingdomlyFeeContract -> ContractNotVerified
```

```
KingdomlyStaking -> InvalidRewardTokenAmount, InvalidWithdrawalAmount
```

Recommendation

Remove the unused error declarations.

Status

Resolved

[Q-02] KingdomlyStaking#depositRewardToken, stake, redeemRewards - Incorrect function descriptions**Description**

The above functions have incorrect descriptions saying the functions can only be called by the owner even if the functions are supposed to be called by anyone.

Recommendation

Update the function descriptions correctly.

Status

Resolved

[Q-03] All contracts - Floating pragma**Description**

The contracts use `pragma solidity ^0.8.24`.

This can allow the contracts to be deployed with the wrong pragma version which is different from the one the contracts were tested with.

Recommendation

Lock the pragma version.

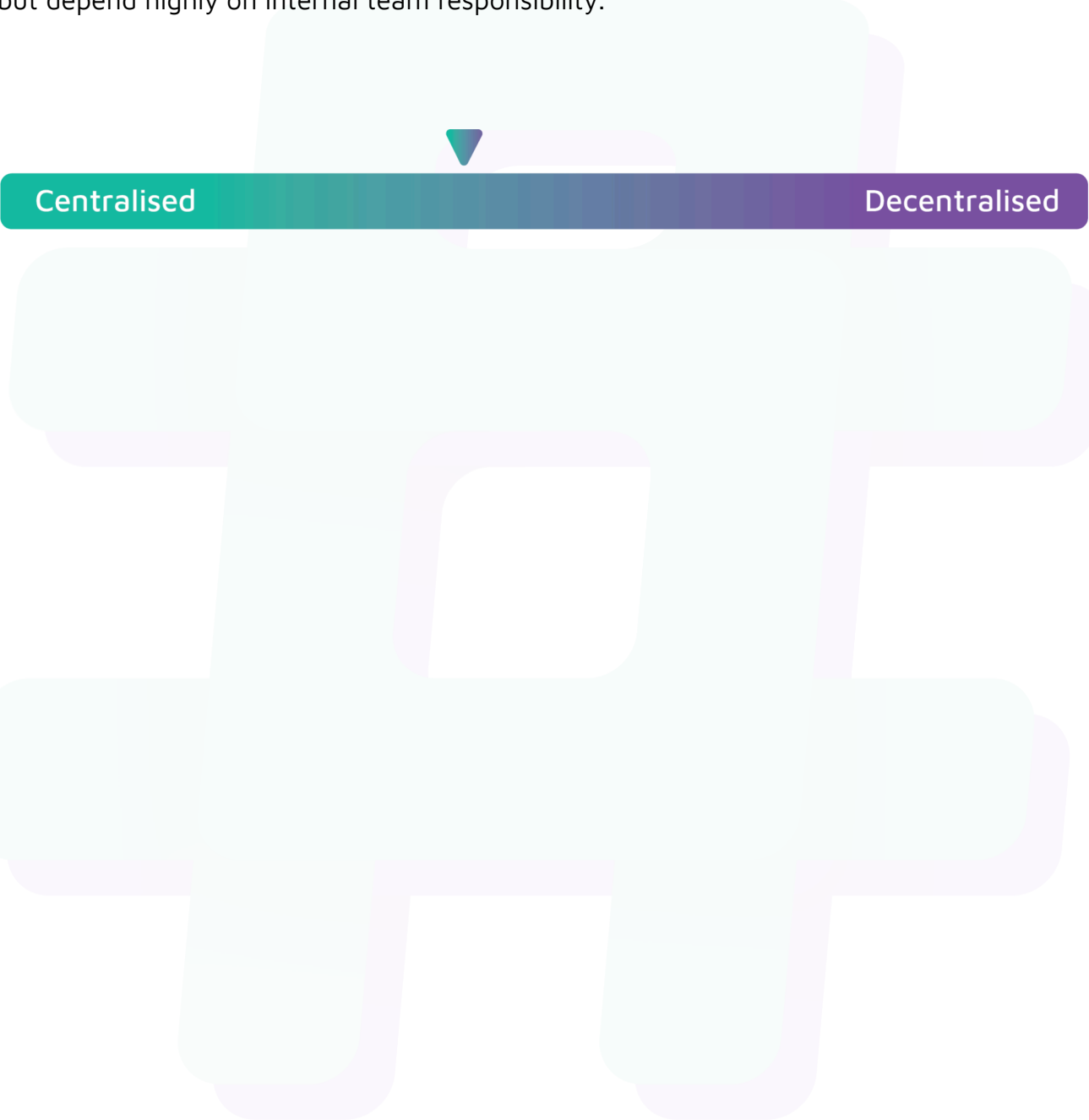
Status

Resolved

Centralisation

The Kingdomly project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Kingdomly project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.